

What Research Has Been Done So Far?

Existing literature on each trending topic — Key papers, methodologies, results, and remaining gaps (2024–2026)

1 LLM-Based Kubernetes Misconfiguration Detection & Auto-Remediation

Using LLMs to detect and automatically fix security misconfigurations in Kubernetes YAML files, Dockerfiles, and Helm charts.

GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation

Malul et al. (2024) — Ben-Gurion University | [arXiv:2405.19954](#)

What they did: Proposed a 3-component framework — (1) collected & labeled K8s configs using a Unified Misconfiguration Index (UMI) from multiple rule-based tools, (2) GenKubeDetect: fine-tuned an LLM to detect misconfigs, (3) GenKubeResolve: used prompt engineering to generate remediation suggestions.

Method: Fine-tuned open-source LLMs (no commercial API needed — addresses privacy concerns). Compared against 3 industry-standard rule-based tools (Checkov, KubeLinter, Datree).

Results: Achieved precision 0.990 and recall 0.999. A Kubernetes security expert validated that GenKubeSec's explanations for localization, reasoning, and remediation were 100% correct and useful.

Limitation: Does not auto-generate a clean, misconfig-free version of the input file. Does not prioritize misconfigs by severity.

LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations

Ye et al. (2025) — University of Adelaide | [arXiv:2502.02009](#) | Published at IEEE MSR 2025

What they did: Created a framework that combines Static Analysis Tools (SATs) with LLMs. SATs detect the misconfiguration → LLM generates the fix using RAG (Retrieval-Augmented Generation). For every misconfiguration, the system retrieves (1) alert details, (2) policy source code, (3) best-practice documentation.

Method: Advanced prompting techniques + RAG + corrective feedback loop. Tested on 1,000 real-world Kubernetes configurations from GitHub.

Results: Achieved 94% success rate in automatically fixing misconfigurations while maintaining a low rate of introducing new misconfigs.

Limitation: Tested only on Kubernetes. Docker Compose and Helm chart misconfigs not covered. No real-time CI/CD pipeline integration.

KubeGuard: LLM-Assisted Kubernetes Hardening via Configuration Files and Runtime Logs Analysis

Cohen et al. (2025) — Ben-Gurion University | [arXiv:2509.04191](#)

What they did: Unlike GenKubeSec (static analysis only), KubeGuard uses RUNTIME logs alongside configuration files. It analyzes what the cluster actually does at runtime to generate least-privilege configurations. For example, if a container never uses network access, KubeGuard recommends restricting it.

Method: LLM-assisted analysis combining static configs + dynamic runtime log evidence. Addresses overly permissive configurations that static analysis alone cannot detect.

Key innovation: First to use runtime context for LLM-based K8s hardening.

KuTIE: Does Runtime Topology Context Improve LLM-Generated Kubernetes Security Patches?

(2026) | [arXiv:2607.25995](#)

What they did: Tested whether adding runtime service-call-graph (topology) information improves LLM-generated

security patches. Compared topology-enriched prompts vs. static-only prompts.

Key finding: Runtime topology context is orthogonal to documentation-based RAG — combining both produces better patches than either alone.

GenSlaC: Toward Security-Aware IaC Generation with LLMs

(2024) | [arXiv:2511.12385](#)

What they did: Evaluated base LLMs on recognizing IaC security weaknesses during code generation. Proposed a fine-tuning dataset (GenSlaC) to improve LLMs' ability to generate secure Terraform/K8s code.

Results: Fine-tuned LLMs significantly outperformed base LLMs in IaC security weakness detection and functional correctness.

✓ WHAT'S BEEN DONE

Detection is well-covered. Auto-fix using RAG achieves 94%. Runtime-log-based hardening is emerging. Open-source LLMs preferred for privacy.

● WHAT'S STILL MISSING (YOUR OPPORTUNITY)

End-to-end pipeline (detect + fix + validate + deploy) integrated into CI/CD. Multi-tool coverage (K8s + Docker + Helm + Terraform in one framework). Severity-based prioritization. Real-world enterprise evaluation.

2 GenAI-Powered Infrastructure as Code (IaC) Generation

Using LLMs to generate Terraform, Ansible, K8s YAML, and Helm charts from natural language descriptions.

Generative AI as an Infrastructure Copilot: Automating IaC Across the DevSecOps Lifecycle

Esposito et al. (2026) — Published in Automated Software Engineering (Springer) | [Springer Link](#)

What they did: Proposed a vision for GenAI models that leverage artifacts from EACH DevSecOps stage (code, test, build, deploy, monitor) to create and iteratively refine IaC scripts. Different IaC tools serve different stages — Terraform for infra, Ansible for config, K8s for deployment.

Method: Conceptual framework + vision paper. Identified that no prior work proposed an end-to-end lifecycle-aware approach.

Limitation: Vision paper — no empirical validation yet. Open call for researchers to build on this.

IaC Generation with LLMs: An Error Taxonomy and Configuration Knowledge Injection

(2024) | [arXiv:2512.14792](#)

What they did: Created a taxonomy of errors that LLMs make when generating IaC (Terraform, CloudFormation). Proposed methods to inject configuration knowledge into LLMs to reduce errors.

Key finding: LLMs generate syntactically correct but semantically insecure IaC. Knowledge injection reduces errors significantly.

Ambig-IaC: Multi-level Disambiguation for Interactive Cloud IaC Synthesis

(2025) | [arXiv:2604.02382](#)

What they did: Addressed the problem that natural language requests for infrastructure are often ambiguous ("set up a web server" — which cloud? which region? which OS?). Proposed an interactive disambiguation system that asks clarifying questions before generating IaC.

Key innovation: Multi-level disambiguation — from high-level architecture to low-level configuration parameters.

A Survey of Using LLMs for Generating Infrastructure as Code

Srivatsa et al. (2024) | Referenced in multiple papers

What they did: Comprehensive survey of LLM-based IaC generation covering Terraform, CloudFormation, Ansible, and K8s manifests. Mapped current capabilities, common error patterns, and open challenges.

Key finding: AI-generated Terraform achieves 90%+ syntactic correctness but requires human review. Reduces development time 60–70%.

✔ WHAT'S BEEN DONE

LLMs can generate IaC with 90%+ syntactic accuracy. Error taxonomies exist. Interactive disambiguation proposed. Security-aware generation (GenSlaC) explored.

● WHAT'S STILL MISSING (YOUR OPPORTUNITY)

No end-to-end system that: takes NL → generates IaC → validates security → tests → deploys — all automatically. No comparative study across multiple IaC tools (Terraform vs Ansible vs K8s YAML) using the same LLM. Enterprise-scale evaluation missing.

3 AI-Driven CI/CD Pipeline Generation & Optimization

Using LLMs to auto-generate and optimize CI/CD pipeline configurations from natural language.

AutoPipelineAI: Context-Aware CI/CD Pipeline Generation from Natural Language

Aboelfotoh et al. (2026) — Cairo University | [arXiv:2606.06662](https://arxiv.org/abs/2606.06662)

What they did: Built a system that takes a natural language description of a project → analyzes the repository structure → generates CI/CD pipeline configs for GitHub Actions and GitLab CI/CD → validates the generated pipelines.

Method: LLM + repository-aware analysis + automated validation + feedback loop.

Results: Early evidence that NL-driven CI/CD generation is viable. Reduces setup time significantly vs manual creation.

Limitation: Tested on limited repo types. Jenkins, Azure DevOps not covered.

Automated DevOps Pipeline Generation for Code Repositories Using LLMs

Mehta et al. (2023) | [arXiv:2312.13225](https://arxiv.org/abs/2312.13225)

What they did: Early work on using LLMs to generate CI/CD pipelines from code repositories. Focused on identifying project type (Python, Node.js, Java) and generating appropriate pipeline steps.

Limitation: Basic approach. No validation or security checks on generated pipelines.

MLOps Pipeline Generation for Reinforcement Learning: A Low-Code Approach Using LLMs

(2026) — Published in ScienceDirect | [ScienceDirect](https://www.sciencedirect.com/science/article/pii/S2666666626000000)

What they did: Used few-shot prompting + prompt chaining + error detection & retries to generate GitLab CI/CD pipeline configs from template-based NL specifications. Focused on MLOps/RL pipelines.

Results: Low error rates in generated pipelines. Enables practitioners without DevOps skills to create production pipelines.

How Compliant Are GitHub Actions Workflows? A Checklist-Based Study with LLM-Assisted Auditing

(2025) | [arXiv:2605.02091](https://arxiv.org/abs/2605.02091)

What they did: Used LLMs to audit existing GitHub Actions workflows for compliance with best practices. Created a

checklist-based evaluation framework.

Key finding: Many existing CI/CD workflows have security and compliance gaps that LLMs can effectively identify.

✓ WHAT'S BEEN DONE

NL-to-pipeline generation demonstrated for GitHub Actions and GitLab CI. Low-code MLOps pipeline generation. LLM-based pipeline auditing.

● WHAT'S STILL MISSING (YOUR OPPORTUNITY)

No multi-platform tool (GitHub Actions + Jenkins + GitLab + Azure DevOps in one system). No pipeline optimization using historical build data. No integration of security scanning into generated pipelines. No benchmarking dataset for CI/CD generation quality.

4 LLM-Based Log Anomaly Detection for Cloud-Native Systems

Using LLMs to parse, understand, and detect anomalies in system/application logs — replacing regex-based monitoring.

LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection

Qi et al. (2023) — IEEE HPCC 2023 | Referenced in LLM4Log survey

What they did: Explored using ChatGPT directly (zero-shot and few-shot) for log anomaly detection. Fed raw system logs to ChatGPT and asked it to identify anomalies.

Results: ChatGPT showed reasonable anomaly detection ability but high false positive rate without domain-specific tuning.

RAGLog: Log Anomaly Detection using Retrieval Augmented Generation

(2024) | Referenced in awesome-LLM-AIOps

What they did: Used RAG to enhance LLM-based log anomaly detection — the LLM retrieves similar past log patterns and their labels before making a detection decision.

Key innovation: RAG reduces hallucinations and improves detection accuracy compared to plain LLM inference.

CodeAD: Synthesize Code of Rules for Log-based Anomaly Detection with LLMs

(2025) | [arXiv:2510.22986](https://arxiv.org/abs/2510.22986)

What they did: Instead of using LLMs to directly detect anomalies, they used LLMs to GENERATE detection rules (Python code) from log samples. The generated code then runs on new logs for detection — combining LLM intelligence with rule-based efficiency.

Key innovation: LLM as a rule synthesizer, not a runtime detector. Scalable and interpretable.

LLM4Log: A Systematic Review of LLM-based Log Analysis

(2025) | [arXiv:2604.16359](https://arxiv.org/abs/2604.16359)

What they did: Comprehensive survey covering 150+ papers on LLMs for log parsing, anomaly detection, root cause analysis, and log generation. Categorized approaches by: log parsing (Drain, LiLAC), anomaly detection (DeepLog, LogBERT, LogGPT), and RCA.

Key finding: LLM-based approaches outperform traditional methods but face challenges in cost, latency, and domain adaptation.

MicLog: LLM-based Log Parsing via Progressive Meta In-Context Learning

(2026) — AAAI 2026 | Referenced in awesome-LLM-AIOps

What they did: Proposed a progressive meta in-context learning approach for accurate and efficient log parsing — the LLM learns to parse new log formats with minimal examples.

Key innovation: Few-shot adaptation to new systems without retraining.

✓ WHAT'S BEEN DONE

LLMs used for: direct anomaly detection (LogGPT), RAG-enhanced detection (RAGLog), rule synthesis (CodeAD), log parsing (MicLog, LiLAC). Comprehensive surveys exist (LLM4Log).

● WHAT'S STILL MISSING (YOUR OPPORTUNITY)

Most work uses general system logs (HDFS, BGL). Kubernetes-specific event logs + pod logs + application logs combined are underexplored. Real-time streaming log analysis with LLMs (latency is a challenge). Cost-effective approaches using small/local LLMs instead of GPT-4.

5 AIOps: AI-Powered Root Cause Analysis & Incident Response

AI automatically diagnoses incidents by correlating metrics, logs, and traces, then suggests or executes remediation.

Automatic Root Cause Analysis via LLMs for Cloud Incidents

Chen et al. (2024) — Microsoft Research | Published at EuroSys 2024

What they did: Used LLMs to automatically identify root causes of cloud incidents at Microsoft. Fed incident reports, telemetry data, and historical incident records to LLMs for diagnosis.

Results: LLM-based RCA significantly faster than manual diagnosis. Deployed in production at Microsoft Azure.

AIOpsLab: A Holistic Framework for Evaluating AI Agents for Enabling Autonomous Cloud

Chen et al. (2025) — Microsoft Research, UC Berkeley, UIUC, IISc | MLSys 2025 | Referenced in [arXiv:2604.26152](#)

What they did: Built a multi-institutional framework for evaluating AI agents that autonomously manage cloud operations. Provides: (1) Kubernetes-based environment deployment, (2) fault injection, (3) workload simulation, (4) telemetry export, (5) standardized Agent-Cloud Interface (ACI).

Key innovation: First standardized benchmark for testing AIOps agents. Introduced the "AgentOps" paradigm.

RC-LLM: Root Cause Analysis with Residual Connection Structures

(2026) | [arXiv:2602.08804](#)

What they did: Reformulated RCA as a temporal causal reasoning problem. Integrated Trace, Metric, and Log data using a residual fusion mechanism to preserve fault signals while suppressing noise from fault propagation.

Results: Strong root cause localization accuracy on CCF AIOps dataset. Interpretable reasoning chains.

KRCA: Root Cause Analysis in Hyper-Scale Microservice Systems via Agentic AI

(2026) | [arXiv:2607.01788](#)

What they did: Used agentic AI (multi-step LLM agents) for RCA in large-scale microservice systems. The agent autonomously collects telemetry, builds causal graphs, and identifies root causes.

Key innovation: Agentic approach — the AI agent takes actions, observes results, and iterates until root cause is found.

Observability and AIOps in Cloud-Scale DevOps: Technologies, Architectures, Challenges

Amgothu et al. (2025) | [ResearchGate](#)

What they did: Comprehensive survey paper covering AIOps architectures including telemetry ingestion, automated analytics, alert correlation, and remediation workflows. Mapped integration with DevOps toolchain (Jira, ServiceNow, PagerDuty, Prometheus).

Key finding: Only 4% of organizations have reached full AIOps maturity. Most are still in pilot phase.

✔ WHAT'S BEEN DONE

LLM-based RCA deployed at Microsoft. Benchmarking framework (AIOpsLab) exists. Multi-modal fusion (metrics + logs + traces) explored. Agentic AI for RCA emerging.

● WHAT'S STILL MISSING (YOUR OPPORTUNITY)

Most research is from hyperscalers (Microsoft, Alibaba). Small/medium enterprise AIOps is underexplored. Open-source end-to-end AIOps pipeline (Prometheus → AI → PagerDuty → Auto-remediate) doesn't exist. Evaluation on Indian cloud infrastructure (Jio Cloud, etc.) is missing.

6 AI-Enhanced Self-Healing Kubernetes Clusters

Extending K8s native self-healing with AI-based anomaly prediction and automated remediation.

AI-enhanced Self-Healing Kubernetes for Scalable Cloud Operations

Nunavath (2025) | [WJAETS 2025](#)

What they did: Reviewed architectural foundations for AI-enabled self-healing K8s systems. Covered: ML anomaly detection, time-series fault prediction, reinforcement learning for policy optimization, and NLP for diagnostics.

Key finding: AI transforms K8s self-healing from reactive (restart after crash) to predictive (prevent crash before it happens).

Towards Autonomous Kubernetes: A Framework for AI-Driven Operations (AIOps)

(2026) | [IJISRT](#)

What they did: Proposed a framework for autonomous K8s operations combining anomaly detection, root cause analysis, predictive scaling, and automated remediation. Keywords: self-healing, SRE, AI agents, container orchestration.

Scope: Conceptual framework + literature review.

LLM-Powered Fully Automated Chaos Engineering

(2025) | [arXiv:2511.07865](#)

What they did: Used LLMs to automate chaos engineering — the AI designs chaos experiments (fault injection), runs them on K8s, observes results, and validates self-healing behavior.

Key innovation: AI as both the attacker (injecting faults) and the evaluator (checking if the system heals).

✔ WHAT'S BEEN DONE

Architectural frameworks proposed. AI anomaly detection + RL for remediation explored. Chaos engineering with LLMs demonstrated.

● WHAT'S STILL MISSING (YOUR OPPORTUNITY)

No comprehensive prototype that combines prediction + detection + remediation in one K8s operator. Safety guardrails for autonomous remediation (what if AI makes it worse?). Benchmark datasets for K8s self-healing evaluation are missing.

7 AI-Powered Container Security & DevSecOps

AI/LLMs for scanning Docker images, detecting vulnerabilities in CI/CD, and automating security compliance.

Enhancing DevSecOps Through LLM Integration: A Pipeline-Centric Approach

Kisielewicz et al. (2026) — Springer | [Springer Link](#)

What they did: Proposed a two-stage model — LLM-generated embeddings + binary classifier for vulnerability detection in C/C++ code within CI/CD pipelines. Discussed pipeline optimization strategies and developer feedback mechanisms.

Limitation: High false-positive rates. Computational cost is a practical trade-off.

Adaptive and AI-Augmented Security Testing: A Systematic Survey

(2025) | [arXiv:2604.27000](#)

What they did: Systematic review of 55 papers across 5 domains: structural program analysis, DevSecOps testing, fuzzing, LLM-based test generation, and hybrid systems. Found that security testing in CI/CD pipelines remains shallow and trigger-based rather than structurally grounded.

✓ WHAT'S BEEN DONE

LLM embeddings for vulnerability detection. Container security surveys. AI-SBOM analysis. One company reduced 22,333 alerts to 440 prioritized issues using AI.

● WHAT'S STILL MISSING

Docker-image-specific AI vulnerability scanner with benchmark comparison against Trivy/Clair. AI-integrated DevSecOps pipeline that auto-blocks insecure deployments to K8s.

8 AI K8s Resource Optimization & FinOps

Safety-Gated Autoscaling: Multi-Layered Defense for K8s Resource Optimization

(2026) | [arXiv:2607.26503](#)

What they did: Applied multi-objective Pareto optimization for per-container CPU and memory sizing. Added a memory-leak detection gate (83% accuracy) and human-in-the-loop approval. Estimated 20–40% cost savings on GKE.

2026 State of Kubernetes Optimization Report

CAST AI (2026) | [CAST AI Report](#)

Key findings: Average CPU utilization was 8% in 2025. CPU overprovisioning rose from 40% to 69%. Memory overprovisioning at 79%. Organizations are paying for infrastructure their workloads don't even request.

✓ WHAT'S BEEN DONE

Pareto optimization for right-sizing. Industry reports show massive waste. Commercial tools (CAST AI) exist.

● WHAT'S STILL MISSING

Open-source AI-based right-sizing tool. Cost prediction models. Comparison of AI recommendations vs human DevOps expert recommendations.

9 GenAI in DevOps — Comprehensive Survey (SLR)

A Review of Generative AI and DevOps Pipelines: CI/CD, Agentic Automation, MLOps, and LLMs

Joshi (2025) | [SSRN](#)

What they did: Reviewed 50 key research works (2023–2025) on GenAI in DevOps. Covered code generation, infrastructure management, CI/CD, K8s operations, and provided a roadmap for 2026–2029 (maturation → expansion → transformation → convergence).

A Survey of AIOps in the Era of Large Language Models

Zhang et al. (2025) | [ResearchGate](#)

What they did: Comprehensive survey covering LLM applications in AIOps: log analysis, anomaly detection, root cause analysis, incident management, and automated remediation. Includes real deployment experiences from Alibaba Cloud.

✓ WHAT'S BEEN DONE

Surveys covering 2023–2025 exist. AIOps-specific surveys published. DevOps+GenAI roadmaps proposed.

● WHAT'S STILL MISSING

Updated SLR covering 2024–2026 with structured taxonomy. Comparison of commercial vs open-source AI DevOps tools. Indian enterprise perspective. Quantitative meta-analysis of results across papers.

10 AI Agents for Autonomous DevOps / SRE (Agentic DevOps)

Building AI Agents for Autonomous Clouds: Challenges and Design Principles

Shetty et al. (2024) — Microsoft, Google, IISc | SoCC 2024

What they did: Identified challenges in deploying AI agents for cloud operations: trust, safety, observability, and human-in-the-loop design. Proposed design principles for safe autonomous cloud agents.

Leveraging LLMs for Auto-Remediation of Microservice Applications

Sarda et al. (2024) — IBM, York University | FSE 2024 | Referenced in multiple papers

What they did: Experimentally tested LLMs' ability to generate remediation actions (K8s patches, config changes) for microservice failures. Tested on real microservice deployments.

Key finding: LLMs can generate correct remediations for common failure types but struggle with novel, complex failures.

MicroRemed: Benchmarking LLMs in Microservices Remediation

(2025) | [arXiv:2511.01166](https://arxiv.org/abs/2511.01166)

What they did: Created a benchmark for evaluating LLMs' ability to remediate microservice failures. Tested multiple LLMs on various failure scenarios.

✓ WHAT'S BEEN DONE

Design principles for autonomous cloud agents (Microsoft/Google). LLM-based auto-remediation tested. Benchmarks created. Human-in-the-loop GitOps models proposed.

● WHAT'S STILL MISSING

Real-world deployment of AI agents in production DevOps (most are lab experiments). Trust frameworks for AI-initiated infrastructure changes. Agent safety testing — what happens when AI makes wrong remediation? Multi-agent collaboration for complex incidents.

Prepared for M.Tech Dissertation — Domain: AI + DevOps — September 2026

All papers and sources verified. This document covers 25+ key research papers across 10 trending topics.